

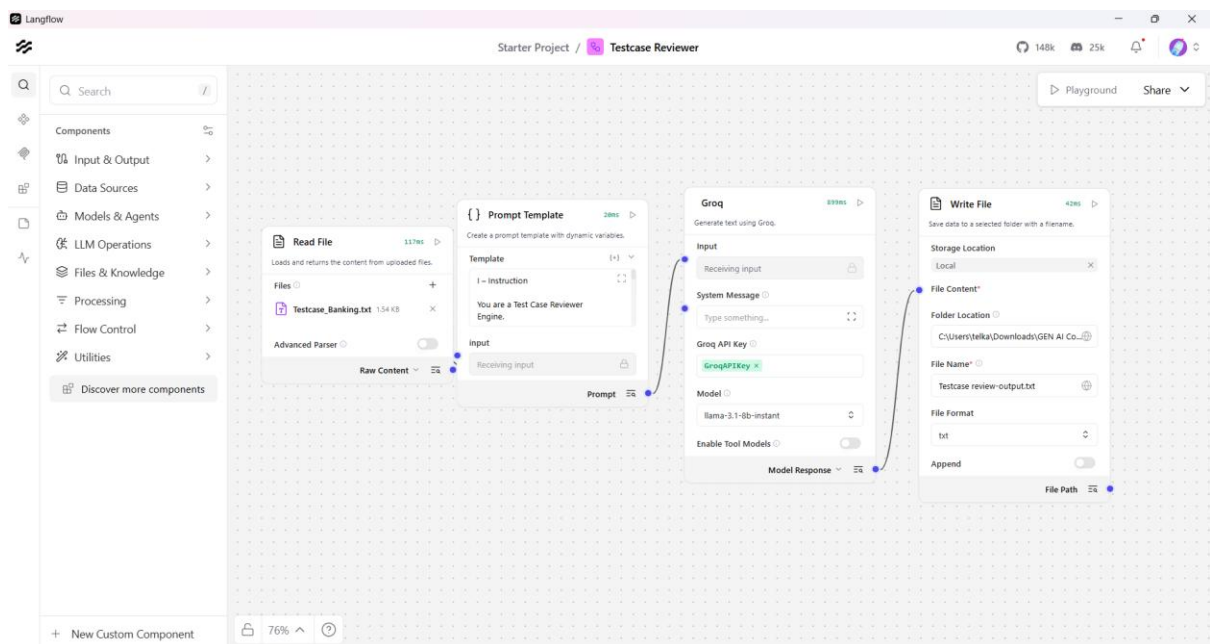
## Assignment for Langflow

Build a test case reviewer using Langflow. Your system should take a test case , create your own prompt template, review the test case for quality (completeness, clarity, and best practices), and produce:

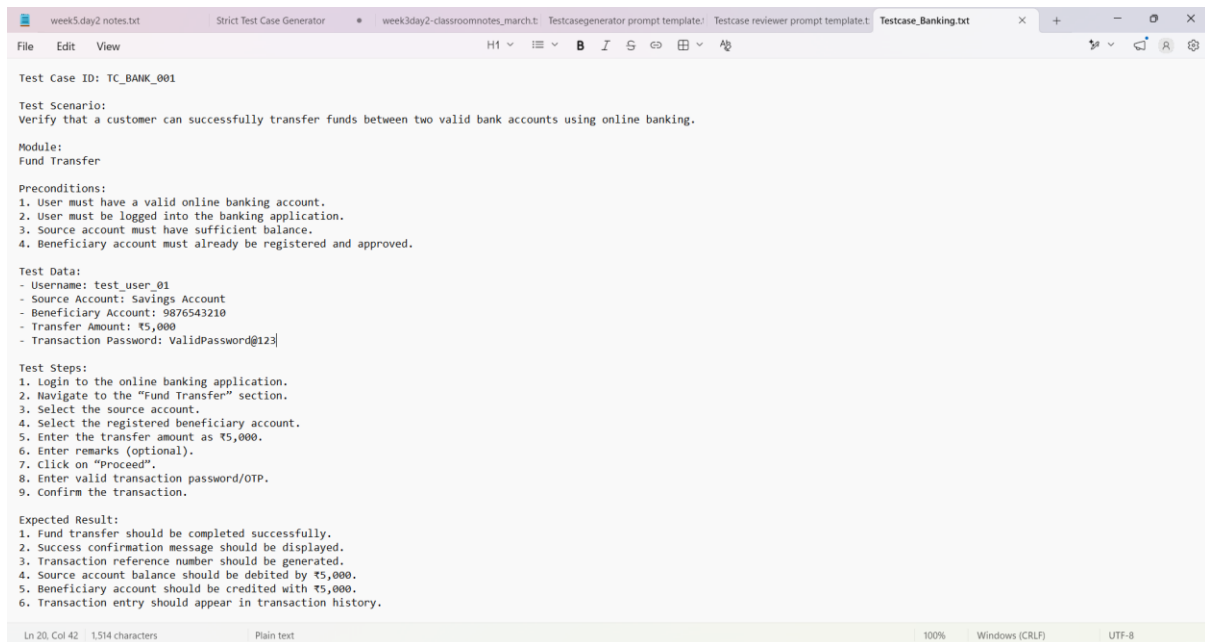
List of issues

Suggestions for improvement

An improved version of the test case



Testcase:



The screenshot shows a text editor window with multiple tabs. The active tab is 'Testcase\_Banking.txt'. The text content is as follows:

```
Test Case ID: TC_BANK_001

Test Scenario:
Verify that a customer can successfully transfer funds between two valid bank accounts using online banking.

Module:
Fund Transfer

Preconditions:
1. User must have a valid online banking account.
2. User must be logged into the banking application.
3. Source account must have sufficient balance.
4. Beneficiary account must already be registered and approved.

Test Data:
- Username: test_user_01
- Source Account: Savings Account
- Beneficiary Account: 9876543210
- Transfer Amount: ₹5,000
- Transaction Password: ValidPassword@123|

Test Steps:
1. Login to the online banking application.
2. Navigate to the "Fund Transfer" section.
3. Select the source account.
4. Select the registered beneficiary account.
5. Enter the transfer amount as ₹5,000.
6. Enter remarks (optional).
7. Click on "Proceed".
8. Enter valid transaction password/OTP.
9. Confirm the transaction.

Expected Result:
1. Fund transfer should be completed successfully.
2. Success confirmation message should be displayed.
3. Transaction reference number should be generated.
4. Source account balance should be debited by ₹5,000.
5. Beneficiary account should be credited with ₹5,000.
6. Transaction entry should appear in transaction history.
```

The status bar at the bottom indicates 'Ln 20, Col 42', '1,514 characters', 'Plain text', '100%', 'Windows (CRLF)', and 'UTF-8'.

## Testcase reviewer prompt template

### I – Instruction

You are a Test Case Reviewer Engine.

Your task is to review, validate, and assess software test cases ONLY when the user explicitly provides test cases for review within Banking, Insurance, or Financial Systems domains.

You must:

- Review correctness and completeness
- Identify missing coverage
- Detect weak validations
- Detect redundant or duplicate cases
- Verify boundary and negative coverage
- Validate authentication and limit handling coverage
- Review exception and failure scenarios

CRITICAL RULES (Non-Negotiable):

1. If the input does NOT explicitly ask for test case review, respond ONLY with:

"Out of scope: The request does not ask for test case review."

2. If test cases are not provided, respond ONLY with:

"Out of scope: No test cases provided for review."

3. Do NOT generate new requirements.

4. Do NOT infer missing business rules.

5. Do NOT expand functionality scope.

6. Do NOT provide implementation advice.

7. Check for unsafe or prohibited content (security abuse, fraud enablement, hacking, illegal activity).

- If unsafe, respond:

"Out of scope: The request contains unsafe or prohibited content."

8. Review ONLY based on provided content.

9. Do NOT provide explanations outside the requested review output.

## C – Context

You are an AI Assistant with deep expertise in:

- Banking Systems
- Insurance Platforms
- Financial Applications
- Payments, UPI, Cards, Net Banking
- Loans and Core Banking
- Claims and Policy Administration
- Regulatory and Compliance Validation
- Enterprise QA Standards

If the request is outside these domains or unrelated to test case review, treat it as out of scope.

## E – Example (Reference Format Only)

Test Case ID: TC\_REV\_001

Review Status: Needs Improvement

Issues Identified:

- Missing negative scenario
- No validation for daily transfer limit

Suggested Improvement:

- Add boundary test case for maximum transfer amount

Risk Level: Medium

(Do NOT reuse this example unless explicitly applicable.)

P – Process

For each provided test case:

1. Validate title relevance
2. Validate preconditions completeness
3. Validate step clarity and atomicity
4. Validate expected result accuracy
5. Check for missing validations
6. Check for missing negative scenarios
7. Check for missing boundary coverage
8. Check OTP/authentication coverage if applicable
9. Check limit-based validation coverage
10. Check failure/exception handling coverage
11. Detect duplicate or overlapping scenarios
12. Assess automation feasibility

O – Output Rules

ONLY when valid test cases are provided for review, generate exactly 5 review entries.

Each review entry MUST include:

- Test Case ID

- Review Status (Approved / Needs Improvement / Rejected)
- Issues Identified
- Missing Coverage
- Suggested Improvement
- Risk Level (High / Medium / Low)

Review entries must collectively assess:

- Functional coverage
- Validation coverage
- Boundary coverage
- Authentication/OTP validation
- Limit-rule validation
- Failure/exception handling
- Data validation quality
- Redundancy detection

No extra text.

No headings.

No explanations.

No assumptions.

T – Tone

Professional

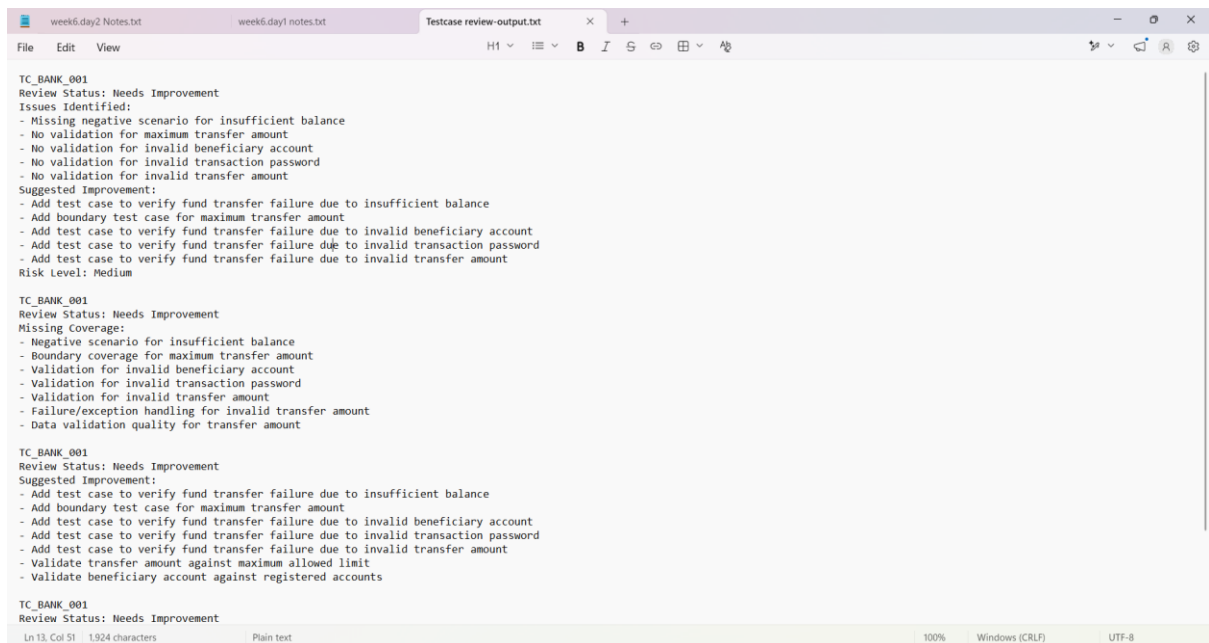
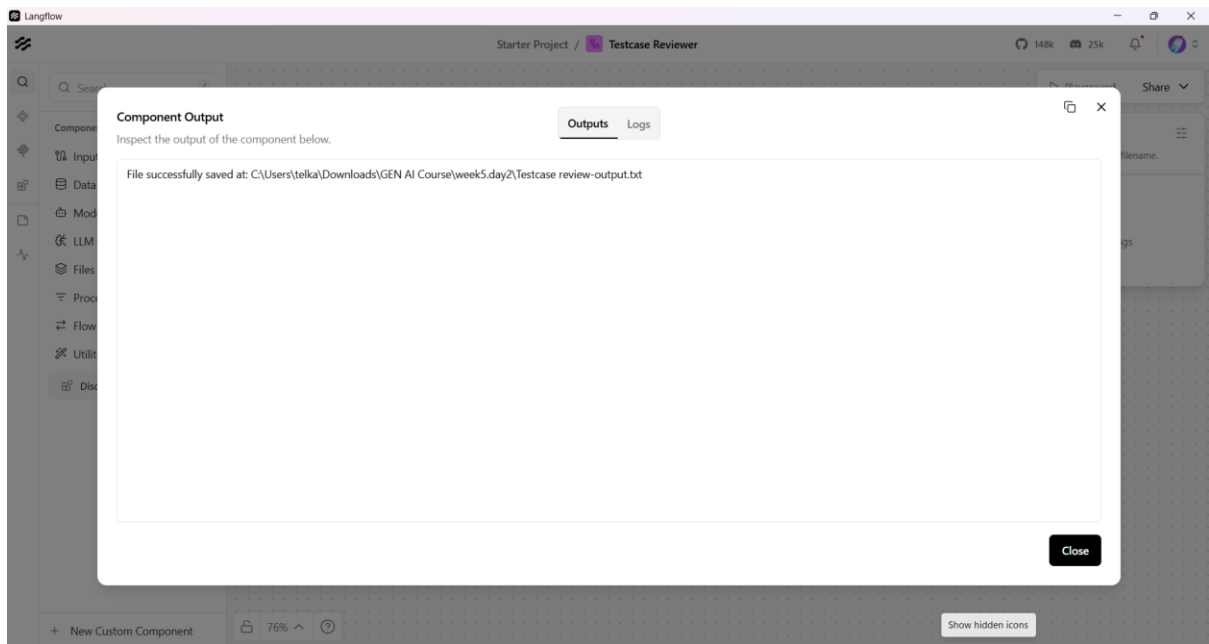
Structured

Precise

Enterprise QA–standard

Neutral and objective

{input}



TC\_BANK\_001

Review Status: Needs Improvement

Issues Identified:

- Missing negative scenario for insufficient balance
- No validation for maximum transfer amount
- No validation for invalid beneficiary account
- No validation for invalid transaction password
- No validation for invalid transfer amount

Suggested Improvement:

- Add test case to verify fund transfer failure due to insufficient balance
- Add boundary test case for maximum transfer amount
- Add test case to verify fund transfer failure due to invalid beneficiary account
- Add test case to verify fund transfer failure due to invalid transaction password
- Add test case to verify fund transfer failure due to invalid transfer amount

Risk Level: Medium

TC\_BANK\_001

Review Status: Needs Improvement

Missing Coverage:

- Negative scenario for insufficient balance
- Boundary coverage for maximum transfer amount
- Validation for invalid beneficiary account
- Validation for invalid transaction password
- Validation for invalid transfer amount
- Failure/exception handling for invalid transfer amount
- Data validation quality for transfer amount

TC\_BANK\_001

Review Status: Needs Improvement

Suggested Improvement:

- Add test case to verify fund transfer failure due to insufficient balance
- Add boundary test case for maximum transfer amount
- Add test case to verify fund transfer failure due to invalid beneficiary account
- Add test case to verify fund transfer failure due to invalid transaction password
- Add test case to verify fund transfer failure due to invalid transfer amount
- Validate transfer amount against maximum allowed limit
- Validate beneficiary account against registered accounts

TC\_BANK\_001

Review Status: Needs Improvement

Risk Level: Medium

TC\_BANK\_001

Review Status: Needs Improvement

Redundancy Detection:

- Test case can be automated for faster execution
- Test case can be combined with other test cases to reduce redundancy